

DATA ENGINEERING PROJECT GUIDELINES

Primary Output	A reproducible, automated, documented end-to-end data engineering pipeline
Minimum Data Sources	1 or more independent sources using at least 1 source types/formats with data volume of 10000 and up (more data sources and more volume, the better)
Required Core Stack	Python, PostgreSQL, Apache Airflow, Docker / Docker Compose, Git/GitHub
Required Data Layers	Raw -> Staging -> Curated
Project Grade	100 points + bonus
Presentation Grade	100 points, including live demonstration and technical defense

Core Principle

The project is evaluated primarily as a Data Engineering system. Data analytics, dashboards, machine learning, or visualizations may add value, but they cannot substitute for missing ingestion, transformation, storage, validation, orchestration, reproducibility, and documentation requirements.

Contents

1. Project Overview
2. Project Objectives
3. Scope and General Expectations
4. Detailed Technical Requirements
5. Required Deliverables
6. Repository and Documentation Standards
7. Project Grading Rubric (100 + 10 Bonus)
8. Presentation and Technical Defense Rubric (100)
9. Technical Defense and Individual Accountability
10. Grading Rules, Penalties, and Score Caps
11. Academic Integrity and Responsible Use of AI
12. Submission Checklist
13. Recommended Presentation Structure
14. Minimum Acceptance Criteria

1. Project Overview

Your team will design and implement an end-to-end Data Engineering project using datasets that are identified, gathered, and approved. The project must begin with a meaningful real-world problem and conclude with a reliable, reproducible data product that can support downstream analytics, reporting, decision-making, or machine learning.

The work must demonstrate the full lifecycle covered in the course: source discovery and profiling, automated ingestion, data quality validation, layered storage, transformation, relational storage, file-format handling, partitioning, orchestration, containerization, logging, configuration management, rerun safety, documentation, and technical communication.

Minimum End-to-End Flow

1+ Data Sources -> Automated Ingestion -> Raw Layer -> Validation -> Staging Layer -> Transformation -> Curated Layer -> PostgreSQL -> Airflow Orchestration -> Dockerized Environment -> Final Data Product -> Documentation -> Presentation

2. Project Objectives

- Formulate a clear, relevant, and technically defensible problem that requires a data engineering solution.
- Gather and integrate data from multiple heterogeneous sources rather than relying on a single pre-cleaned dataset.
- Design an appropriate end-to-end data architecture and justify the selected technologies and design decisions.
- Build an automated, modular, reproducible, and rerun-safe ingestion and transformation pipeline.
- Apply data quality checks, schema controls, error handling, logging, and operational safeguards.
- Use PostgreSQL for structured storage and demonstrate appropriate data modeling, keys, relationships, and querying.
- Use CSV, JSON, and Parquet appropriately and demonstrate understanding of their storage and performance trade-offs.
- Orchestrate the workflow with Apache Airflow and package the execution environment using Docker / Docker Compose.
- Produce complete technical documentation so that another technically competent person can reproduce and operate the project.
- Defend the architecture, implementation, limitations, and engineering decisions during a live presentation and Q&A.

- Optionally extend the curated data product with meaningful Data Analytics/ Science for bonus credit.

3. Scope and General Expectations

The project must be substantial enough to require integration, transformation, storage, validation, and automation. A notebook that downloads one dataset (with limited number of data) and performs exploratory analysis is not considered a complete Data Engineering project.

- The problem should be realistic and should have identifiable users, stakeholders, or downstream consumers.
- The final pipeline must be executable using the documented environment and instructions.
- All mandatory technical components must be present in the repository and must correspond to the architecture shown in the documentation.
- Raw source data must remain traceable to its origin. Destructive manual preprocessing before ingestion should be avoided.
- Automation is expected. Repetitive manual copying, cleaning, or loading of data will reduce the grade.
- Notebooks may be used for exploration, profiling, validation experiments, and optional analytics; however, the production pipeline must be modularized into scripts/modules.
- Every team member must understand the major components of the project and may be asked to explain or demonstrate any part during the presentation.

4. Detailed Technical Requirements

4.1 Problem Definition and Use Case

- Provide the project title, problem statement, objectives, target users/stakeholders, expected data product, scope, and limitations.
- Explain why the problem requires a data pipeline rather than a one-time analysis.
- Identify the key questions, decisions, or downstream use cases the final curated dataset should support.

4.2 Dataset Collection and Source Diversity

- Use at least one (1) independent data sources.
- Use at least two (2) different source types or formats. Examples include CSV, JSON, Parquet, REST API, relational database, public open-data portal, or another instructor-approved structured/semi-structured source.
- (Plus points) At least one source must be retrieved programmatically, such as through a REST API, database query, or automated file retrieval process.
- A total of at least 10,000 records after ingestion is recommended. Smaller datasets require a clear domain-specific justification.
- Document the provider, source URL or endpoint, access date, format, update frequency (if known), license/usage restrictions, retrieval method, and known limitations for each source.

4.3 Source Inventory and Data Profiling

- Profile every source before transformation.
- Include row/column counts, field names, data types, missingness, duplicates, unique values, date ranges, invalid values, inconsistent formats, basic descriptive statistics, and notable data-quality issues.
- Summarize source limitations and risks that may affect integration or downstream use.

4.4 Data Engineering Architecture

- Create an architecture diagram showing sources, ingestion, raw storage, validation, staging, transformation, curated storage, PostgreSQL, orchestration, and the consumption/analytics layer.
- Explain the role of each major component and justify the chosen tools and technologies.
- The documented architecture must accurately represent the implemented system.

4.5 Automated Data Ingestion

- Implement programmatic extraction/loading for the selected sources.
- Preserve source/raw data as close to the original form as practical.
- Record ingestion metadata such as source, retrieval timestamp, or batch identifier where appropriate.
- Handle common extraction failures, invalid responses, missing files, or connection errors.

4.6 Raw, Staging, and Curated Layers

- Raw Layer: retain source-faithful data and preserve traceability.
- Staging Layer: clean, standardize, validate, and prepare data for integration.
- Curated Layer: produce integrated, analysis-ready or consumption-ready datasets/tables.
- Clearly document what transformations are allowed or performed in each layer.

4.7 Data Transformation and Integration

- Perform meaningful cleaning, type conversion, standardization, missing-value treatment, duplicate handling, filtering, joins, aggregations, derived-field creation, and business-rule implementation as appropriate.
- Use reusable functions/modules rather than duplicating logic across scripts.
- Document important transformation rules and assumptions.

4.8 Data Quality Validation

- Implement at least five (5) types of automated data-quality checks.
- Possible checks include schema, data type, nullability, uniqueness, duplicates, accepted values, ranges, dates, referential integrity, row counts, or domain/business rules.
- Validation failures should be visible through logs, exceptions, task failures, or validation reports.

4.9 PostgreSQL Storage and Data Modeling

- Use PostgreSQL as a required structured storage component.
- Design tables using appropriate field types, primary keys, foreign keys, and relationships where applicable.
- Provide SQL schema/DDDL scripts and demonstrate loading and retrieval of processed data.
- Provide an ERD or database schema diagram and representative SQL queries.

4.10 CSV, JSON, and Parquet Handling

- Demonstrate the use of CSV, JSON, and Parquet within the project workflow.
- Compare at least file size and read/write performance where practical.
- Discuss schema preservation, analytical suitability, and trade-offs among the formats.

4.11 Data Partitioning

- Implement an appropriate partitioning strategy, such as by year, month, region, category, or another meaningful field.
- Demonstrate reading or processing selected partitions rather than always scanning all data.
- Explain why the chosen partition key is appropriate.

4.12 Apache Airflow Orchestration

- Create an Airflow DAG representing the major pipeline stages.
- Configure task dependencies, scheduling, parameters/configuration, retries, and failure handling.
- Demonstrate successful Airflow runs and show how failures can be diagnosed from task logs.
- The Airflow DAG code must be committed to the repository; screenshots alone are insufficient.

4.13 Dockerization and Environment Reproducibility

- Provide a Dockerfile and docker-compose.yml (or equivalent Compose file).

- Containerize the required services, including PostgreSQL and the pipeline environment; include Airflow as applicable.
- Document how the complete environment can be started and stopped.
- Another evaluator should be able to reproduce the project using the repository instructions with minimal undocumented setup.

4.14 Configuration and Secrets Management

- Externalize configuration such as database host, database name, ports, paths, and API credentials.
- Provide a .env.example or equivalent configuration template.
- Do not commit actual passwords, tokens, API keys, private certificates, or other secrets.
- Avoid hard-coded machine-specific paths and credentials.

4.15 Error Handling and Logging

- Use appropriate try/except logic and meaningful exceptions.
- Log major stages, record counts, warnings, failures, and successful loads.
- Logs should be useful for diagnosing failures rather than merely printing generic messages.

4.16 Idempotency and Rerun Safety

- Design the pipeline so that repeated execution does not unintentionally duplicate or corrupt data.
- Use an appropriate strategy such as UPSERT, merge, deduplication, replace-partition, batch identifiers, or controlled truncate-and-load.
- Explain the chosen rerun strategy in the documentation and demonstrate it where practical.

4.17 Modular Python Implementation

- Separate extraction, transformation, loading, validation, configuration, and utility responsibilities.
- Use reusable functions/classes/modules and meaningful file names.
- Do not place the entire production pipeline in a single Jupyter notebook.

4.18 Git and GitHub

- Maintain the project in a GitHub repository with a meaningful structure and commit history.
- Include .gitignore and exclude secrets, local environments, caches, and unnecessary generated files.
- For group projects, each member should make substantive contributions that can be verified through commits, code ownership, documentation, or the technical defense.

4.19 Data Dictionary

- Provide a data dictionary for the final curated dataset(s) or tables.
- Include at least field name, data type, description, nullability, and representative example or allowed-value information where appropriate.

4.20 Data Contract

- Provide at least one formal data contract for an important dataset or table.
- Define expected fields, types, required/optional status, nullability, constraints, and validation rules.
- Identify the producer/source and intended consumer where appropriate.

4.21 Required Technical Diagrams

- Provide a Data Engineering architecture diagram.
- Provide a data flow or lineage diagram showing movement and transformation of data.
- Provide a database ERD/schema diagram.

4.22 Final Curated Data Product

- Produce one or more clean, integrated, validated, consumption-ready outputs.
- The final data product must directly support the original problem statement and stated objectives.
- Document its schema, refresh/process logic, and intended downstream use.

4.23 Final Technical Report

- Include the problem and objectives, sources, architecture, implementation, data model, transformations, validation, orchestration, deployment, results, challenges, limitations, and future improvements.
- The report should explain decisions and trade-offs, not merely repeat screenshots or code.

4.24 Presentation and Live Demonstration

- Demonstrate the architecture, source ingestion, pipeline execution, Airflow, PostgreSQL, final curated output, and relevant logs or validation results.
- Be prepared to execute or inspect code live, trace a record through the pipeline, and answer technical questions.
- Screenshots may support the presentation but do not replace a functioning implementation.

4.25 Optional Bonus: Data Analytics

- Use the curated data product for meaningful analytics such as EDA, dashboarding, statistical analysis, forecasting, anomaly detection, geospatial analysis, machine learning, or another justified analytical method.
- Analytics must produce defensible insights related to the original problem.
- Bonus work does not replace any mandatory Data Engineering requirement.

5. Required Deliverables

#	Deliverable	Minimum Expectation
1	GitHub Repository	Complete source code, configurations, documentation, SQL, Airflow DAGs, and project assets.
2	README.md	Complete setup, execution, architecture, source, and usage documentation.
3	Problem Statement & Objectives	Formal statement of the problem, stakeholders, objectives, scope, and expected data product.
4	Source Inventory & Profiling Report	Source metadata, profiling outputs, quality issues, and limitations.
5	Architecture Diagram	End-to-end Data Engineering architecture.
6	Data Flow / Lineage Diagram	Movement and transformation of data across pipeline stages.
7	ERD / Database Schema Diagram	PostgreSQL entities, keys, and relationships.
8	Ingestion Code	Automated source extraction/loading scripts.
9	Transformation Code	Raw → staging → curated processing logic.
10	Validation Code	Automated data-quality checks and validation outputs.
11	PostgreSQL Implementation	DDL/schema scripts, loading logic, and representative queries.
12	CSV / JSON / Parquet Outputs	Evidence of required file-format handling and comparison.
13	Partitioned Dataset	Partitioned data and demonstration of selected-partition access.
14	Airflow DAG	Orchestration code, dependencies, scheduling, retries, and failure handling.
15	Dockerfile + Docker Compose	Reproducible execution environment and required services.
16	.env.example / Configuration Template	Documented environment/configuration variables without real secrets.

#	Deliverable	Minimum Expectation
17	Data Dictionary	Field-level documentation for curated output.
18	Data Contract	Formal schema/quality expectations for at least one key dataset.
19	Final Technical Report	Complete technical narrative, decisions, results, challenges, and limitations.
20	Presentation Slides	Concise technical presentation aligned to the implemented project.
21	Live Demonstration / Defense	Functional demonstration and individual technical Q&A.
22	Optional Analytics Artifact	Dashboard, notebook, report, model, or other analytics output for bonus credit.

6. Repository and Documentation Standards

The repository should be organized so that an evaluator can locate, run, and understand each pipeline component without guessing. The following structure is recommended; equivalent structures are acceptable if responsibilities remain clear.

```

project/
├── README.md
├── requirements.txt / pyproject.toml
├── .gitignore
├── .env.example
├── Dockerfile
├── docker-compose.yml
├── config/
├── dags/
├── data/
│   ├── raw/
│   ├── staging/
│   └── curated/
├── docs/
│   ├── architecture.*
│   ├── data_flow.*
│   ├── erd.*
│   ├── data_dictionary.*
│   └── data_contract.*
├── notebooks/
├── src/
│   ├── extract/
│   ├── transform/
│   ├── load/
│   ├── validation/
│   └── utils/
├── sql/
├── tests/
└── outputs/

```

6.1 README Minimum Contents

- Project overview and problem statement
- Objectives and scope
- Team members / roles, if applicable
- Data source inventory
- Architecture diagram and technology stack
- Repository structure
- Installation and prerequisites
- Configuration and environment variables
- How to start Docker services
- How to initialize PostgreSQL
- How to run the pipeline

- How to run / inspect Airflow
- Data quality and validation approach
- Expected outputs and where to find them
- Known limitations and assumptions
- Troubleshooting notes
- Future improvements

7. Project Grading Rubric (100 Points + 10 Bonus)

The base Data Engineering Project is graded out of 100 points. Optional Data Analytics can earn up to 10 additional bonus points. The bonus is assessed separately and does not compensate for missing mandatory Data Engineering components.

Criterion	Pts	Full-Credit Indicators	Common Reasons for Partial / Low Credit
Problem Definition & Use Case	8	Clear, specific, realistic problem; stakeholders, objectives, expected output, and need for a pipeline are well justified.	Vague/trivial problem, weak stakeholder connection, or poor alignment between problem and implemented pipeline.
Dataset Gathering & Source Selection	10	3+ meaningful independent sources, 3+ source types/formats, programmatic retrieval, complete provenance and justification.	Insufficient source diversity, undocumented/manual sources, overly pre-cleaned data, or weak justification.
Source Profiling & Understanding	7	Comprehensive profiling of schema, types, nulls, duplicates, ranges, distributions, dates, quality issues, and limitations.	Basic or incomplete profiling; quality issues not investigated or documented.
Architecture & Design	10	Complete end-to-end architecture accurately matches implementation; tools, layers, and trade-offs are justified.	Architecture is incomplete, decorative, or inconsistent with the actual system.
Data Ingestion / Extraction	10	Automated, modular, configurable ingestion with raw preservation, metadata, and source failure handling.	Substantial manual steps, brittle scripts, missing raw preservation, or incomplete automation.
Transformation & Processing	10	Reusable raw→staging→curated logic with meaningful cleaning, integration, aggregation, and documented rules.	Minimal or notebook-only processing, duplicated logic, weak transformations, or incorrect integration.
Data Quality & Validation	8	5+ meaningful automated checks integrated into the pipeline with visible outcomes/failures.	Few, superficial, manual, or non-enforced checks.
Storage & Data Modeling	8	Appropriate PostgreSQL design, keys/relationships, SQL scripts, ERD, queries, and sensible file-format/storage choices.	Poor schema design, missing constraints, weak relational modeling, or incomplete database implementation.
Pipeline Orchestration	8	Functional Airflow DAG with dependencies, schedule, parameters, retries, logging, and failure handling.	Partial DAG, manual orchestration, screenshots without code, or nonfunctional dependencies.
Reliability, Error Handling & Rerun Safety	5	Meaningful logging/exceptions and a demonstrated idempotent or controlled rerun strategy.	Fragile pipeline, silent failures, duplicate-producing reruns, or poor diagnostic logging.
Reproducibility, Environment & Deployment	5	Dockerized and documented environment; externalized configuration; reproducible setup on another machine.	Machine-specific setup, undocumented dependencies, hard-coded paths, or incomplete containerization.
Documentation & Technical Communication	8	README/report fully explain sources, architecture, setup, execution, schemas, contracts, assumptions, limitations, and outputs.	Incomplete, inconsistent, outdated, or non-reproducible documentation.
Code Quality, Git & Repository Organization	3	Clean modular code, sensible repository structure, .gitignore, meaningful commits, and no secrets.	Disorganized code, weak version control, generated clutter, poor naming, or unsafe repository practices.
TOTAL BASE PROJECT	100		

7.1 Optional Data Analytics Bonus (+10)

Bonus Area	Maximum
Meaningful EDA using curated data	+2
Effective visualization or dashboard	+2
Appropriate statistical / descriptive analysis	+2
Insights directly answer the project problem	+2
Advanced analytics such as forecasting, ML, anomaly detection, geospatial analysis, or equivalent	+2

8. Presentation Rubric (100 Points)

Criterion	Pts	Full-Credit Indicators	Common Reasons for Partial / Low Credit
Problem & Project Motivation	8	Problem, stakeholders, relevance, and expected value are immediately clear.	Unclear motivation or weak connection to the engineering solution.
Dataset & Source Explanation	8	Sources, scale, acquisition methods, structures, limitations, and selection rationale are accurately explained.	Superficial or inaccurate description of the datasets.
Architecture Explanation	15	Team clearly traces the complete architecture and data movement and justifies key design choices.	Diagram is shown but not understood; missing components or weak justification.
End-to-End Pipeline Demonstration	15	Convincing live evidence from ingestion through final output, including Airflow/PostgreSQL where applicable.	Partial demo, reliance on screenshots, or failure to show operational pipeline behavior.
Technical Depth & Engineering Decisions	12	Strong explanation of transformations, storage, schemas, partitioning, orchestration, configuration, reliability, and trade-offs.	Surface-level narration without technical reasoning.
Data Quality, Reliability & Challenges	8	Explains validation, failures, limitations, logging, recovery, and engineering solutions.	Challenges are hidden, unexplained, or quality controls are poorly understood.
Results & Final Data Product / Analytics	8	Curated output clearly addresses the original problem; bonus analytics, if any, are properly interpreted.	Outputs are unclear, disconnected from the problem, or overclaim analytical results.
Technical Q&A / Defense	15	Answers demonstrate individual ownership and understanding of architecture, code, data, assumptions, and limitations.	Cannot explain implementation details, relies heavily on teammates, or gives memorized/nontechnical answers.
Presentation Structure & Communication	6	Logical, concise, technically precise flow with appropriate pacing and terminology.	Disorganized, overly verbose, unclear, or difficult to follow.
Slide / Visual Quality	3	Diagrams, figures, and tables are legible, purposeful, and support technical explanation.	Crowded, unreadable, decorative, or inconsistent visuals.
Time Management & Team Participation	2	Stays within allotted time and all members make substantive contributions.	Major timing issues or visibly unbalanced participation.
TOTAL PRESENTATION	100		

9. Technical Presentation and Individual Accountability

The presentation is not only a reporting exercise; it is also a technical defense. Any student may be asked to inspect code, explain a design decision, trace data across layers, diagnose a failure, or justify a technology choice.

10. Grading Rules, Penalties, and Score Caps

Condition	Grading Treatment
Pipeline cannot run / no convincing operational evidence	Project score may be capped at 60/100.
No actual automated ingestion	Project score may be capped at 65/100.
No meaningful transformation pipeline	Project score may be capped at 60/100.
Production implementation exists almost entirely in notebooks	Project score may be capped at 75/100.
Documentation is insufficient to reproduce the project	Project score may be capped at 75/100.
Manual cleaning occurs before data enters the pipeline	Deduct approximately 5–15 points depending on severity and impact.
Hard-coded credentials / API keys / passwords	Deduct approximately 5 points and require correction.
Actual secrets committed to GitHub	Deduct approximately 5–10 points; credentials must be revoked/rotated and removed.
Absolute, machine-specific paths prevent reproduction	Deduct approximately 3–8 points depending on severity.
Architecture diagram materially differs from implementation	Deduct approximately 2–5 points and reduce Architecture score accordingly.
Missing required core technology/component	Receive zero or substantial deduction in the corresponding rubric category; severe omissions may also trigger an overall completeness cap.
Bonus analytics is strong but Data Engineering is incomplete	Analytics bonus cannot replace or recover missing core Data Engineering requirements.

11. Academic Integrity and Responsible Use of AI

- All external datasets, code, libraries, tutorials, repositories, diagrams, and substantial technical references must be properly acknowledged where appropriate.
- Students must be able to explain and defend all submitted code, including code produced with AI-assisted tools.
- Using AI tools for brainstorming, debugging, explanation, code assistance, or documentation support does not remove the student's responsibility to verify correctness, security, and reproducibility.
- Submitting copied code or another team's implementation as original work is subject to the Mapua's academic integrity policy.
- Fabricated datasets, fabricated results, fabricated pipeline runs, or manipulated screenshots intended to represent functionality that does not exist are serious integrity violations.
- Credentials, personal data, confidential information, and restricted datasets must not be exposed in the repository or presentation.

12. Submission Checklist

Done	Requirement
☐	Problem statement and objectives are complete.
☐	At least 3 independent data sources are documented.
☐	At least 3 source types/formats are used.
☐	At least one source is retrieved programmatically.
☐	Source profiling is complete.

Done	Requirement
☐	Architecture diagram matches the implementation.
☐	Raw, staging, and curated layers are present.
☐	Automated ingestion works.
☐	Transformation/integration pipeline works.
☐	At least 5 automated data-quality checks are implemented.
☐	PostgreSQL schema, load, and queries work.
☐	CSV, JSON, and Parquet are demonstrated.
☐	Partitioning is implemented and explained.
☐	Airflow DAG executes with correct dependencies/retries.
☐	Dockerfile and Docker Compose are present and usable.
☐	Configuration is externalized; .env.example is provided.
☐	No secrets are committed.
☐	Logging and error handling are meaningful.
☐	Rerun safety/idempotency is implemented and explained.
☐	Code is modularized outside notebooks.
☐	GitHub repository is organized with meaningful commits.
☐	Data dictionary is complete.
☐	Data contract is complete.
☐	Data flow/lineage and ERD diagrams are included.
☐	Final curated data product is available.
☐	README enables reproduction of the project.
☐	Final technical report is complete.
☐	Presentation slides are ready.
☐	Live demo has been tested.
☐	Each member is prepared for individual technical Q&A.
☐	Optional analytics artifact is included only if claiming bonus credit.

13. Recommended Presentation Structure

Order	Presentation Content
1	Problem, stakeholders, objectives, and expected data product
2	Data sources and source characteristics
3	Source profiling and major data-quality issues
4	End-to-end architecture

5	Automated ingestion and raw layer
6	Staging, transformations, and curated layer
7	Data-quality validation
8	PostgreSQL schema / ERD and representative queries
9	CSV / JSON / Parquet and partitioning decisions
10	Airflow orchestration, retries, and failure handling
11	Docker / environment and configuration management
12	Logging, error handling, and rerun safety
13	Final curated data product
14	Optional Data Analytics and insights
15	Challenges, limitations, and future improvements
16	Live demonstration
17	Technical defense / Q&A

14. Minimum Acceptance Criteria

Minimum Passing Technical Implementation

At minimum, the project should demonstrate: data sources -> automated ingestion -> raw layer -> staging transformation -> curated layer -> PostgreSQL storage -> automated validation -> Airflow orchestration -> Dockerized execution -> reproducible GitHub documentation.

A stronger submission will additionally demonstrate thoughtful partitioning, efficient file-format use, incremental or controlled loading, robust idempotency, reusable validation, meaningful logs, sound schema design, data contracts, clear lineage, performance awareness, and an analytics-ready final data product.

The central grading question is whether the submission behaves like a credible Data Engineering system: reproducible, automated, traceable, validated, maintainable, and technically defensible.

15. Important Schedule

Date	Deliverable
October 2, 2026	Deadline of Submission: Project Presentation Document
October 6-8, 2026	Project Presentation
October 14, 2026	Final Paper Deadline

16. Group Presentation Schedule

Date	Deliverable
October 6, 2026	IOTA, KAPPA
October 7, 2026	EPSILON, ZETTA, ETA, THETA, GAMMA, DELTA
October 8, 2026	ALPHA, BETA

17. Group Presentation Guidelines

- Each group will have 20 minutes to present their corresponding project
- 15 minutes will be allocated for Q&A
- Reflect all the feedback on the presentation on the academic paper
- The whole class will provide an anonymous feedback and grade for the presenters

— End of Project Guidelines —